Performance Evaluation of a Parallel Conjugate Gradient Solver

An HPCG-inspired benchmark · HPC Final Lab Project — P08

[Team member names] · [College / Department] · 2026

Abstract

We built a benchmark inspired by HPCG that solves a large sparse system of equations with the Conjugate Gradient (CG) method, in four versions: sequential, OpenMP, MPI and hybrid MPI+OpenMP. All versions give the same correct answer. On a problem with 2.1 million unknowns, the best speedup on 8 cores was **1.40×**. We show that this limit comes from memory bandwidth, not from the code, and that the way MPI processes communicate changes performance by up to **1.69×**.

2.1 M	296	1.40×	10 / 10
unknowns solved	iterations, all versions	best speedup (8 cores)	tests passed

1 Objective

Supercomputers are usually ranked with HPL, which measures arithmetic speed. Real scientific programs are often limited by how fast data moves from memory instead. HPCG was introduced to measure this [1]. Our goal was to implement the core of such a benchmark and measure how different parallel programming methods perform on it.

2 Method

Problem. We solve **Ax = b**, where A comes from the 3D Poisson equation on a grid. Each grid point is linked to its 6 neighbours (value 6 on the diagonal, −1 for each neighbour). The matrix is very sparse (over 99.99% zeros), so only the non-zero values are stored, in CSR format. We set b = A·1, so the correct answer is known to be all ones.

Algorithm — Conjugate Gradient. Starting from a guess, CG improves the solution step by step:

$$\alpha = (r \cdot r) \,/\, (p \cdot Ap) \;\rightarrow\; x = x + \alpha p \;\rightarrow\; r = r - \alpha Ap \;\rightarrow\; p = r + \beta p$$

where r = b − Ax is the remaining error. It stops when the relative error ||r||/||b|| falls below 10^{-8}. Each step needs one sparse matrix–vector product and two dot products.

Performance measures.

$$\text{Speedup } S = T_{\text{sequential}} \,/\, T_{\text{parallel}} \qquad \text{Efficiency } E = S \,/\, p \times 100\%$$

3 Implementations

Version	How it works
Sequential	One core. Used as the baseline.
OpenMP	Threads share memory; loops are split between threads.
MPI	Separate processes, each owns part of the rows. Two ways to share data: **Allgatherv** (everyone receives the whole vector) and **halo exchange** (only neighbouring values).
Hybrid	MPI processes, each using OpenMP threads.

Correctness was checked with 10 automated tests (matrix properties, exact solution, and identical results across all versions, including MPI with 1–4 processes). All tests pass.

4 Experimental Setup

Hardware	Intel Core 7 240H laptop, 16 logical CPUs
Software	C++17 (g++ 13.3, -O3), OpenMP 4.5, Open MPI 4.1.6, Ubuntu 24.04 (Docker)
Problem	128 × 128 × 128 grid = 2,097,152 unknowns, 14.6 million non-zeros
Measurement	1 warm-up + 5 timed runs, median reported; matrix setup not timed

5 Results

Every version needed exactly **296 iterations** and reached the same accuracy (error from the exact answer $\approx 7.7\times10^{-8}$), so all timings below compare the same computation.

Configuration	Cores	Time (s)	Speedup	Efficiency	Comm. time
Sequential	1	5.95	1.00	100%	–
OpenMP	4	4.45	1.34	33%	–
OpenMP (best)	**8**	**4.24**	**1.40**	**18%**	**–**
MPI – Allgatherv	8	7.45	0.80	10%	51%
MPI – halo exchange	8	4.40	1.35	17%	12%
Hybrid (4 × 2)	8	4.27	1.39	17%	5%

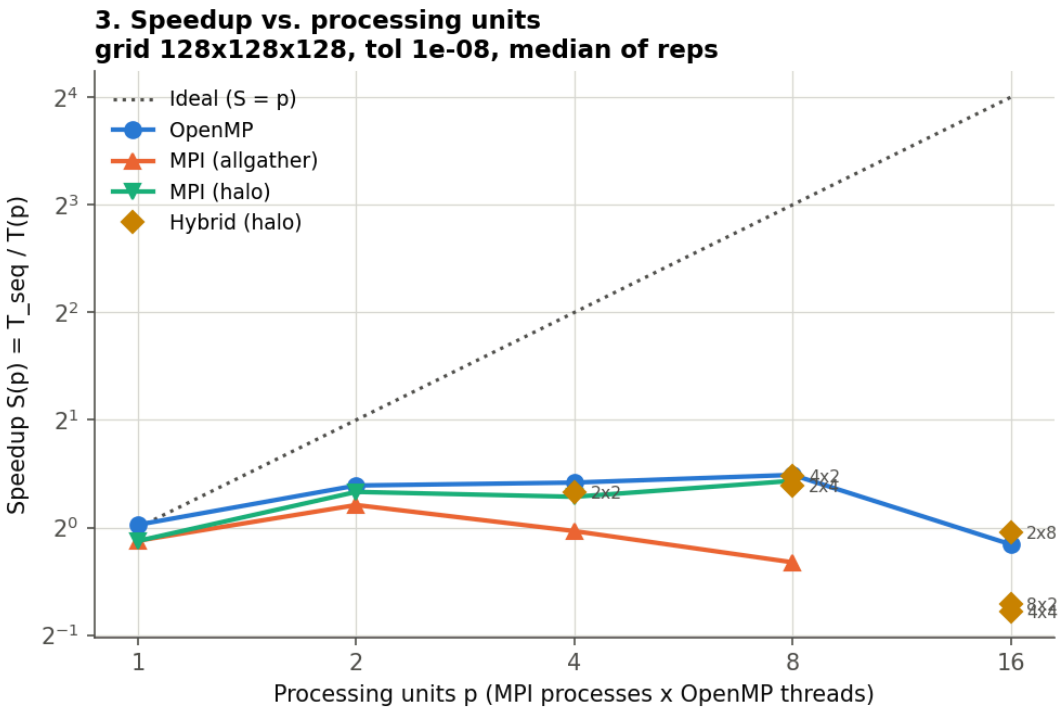


Figure 1. Speedup vs number of cores. The dotted line is the ideal.

> All versions level off at about 1.4×. Adding more cores beyond this point does not help.

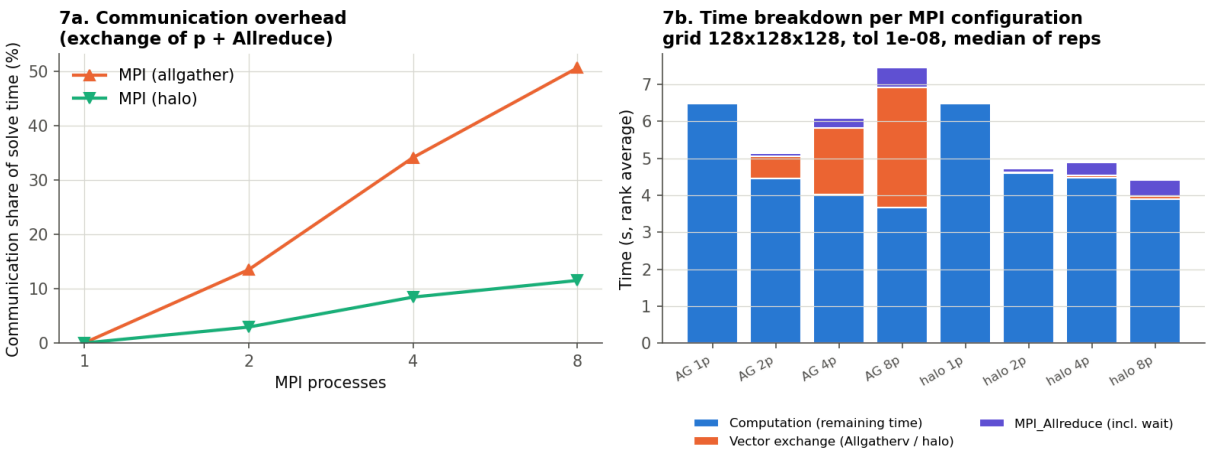


Figure 2. Share of time spent on communication between MPI processes.

> Allgatherv spends 51% of its time communicating at 8 processes and becomes slower than one core. Halo exchange stays around 12% and is **1.69× faster**.

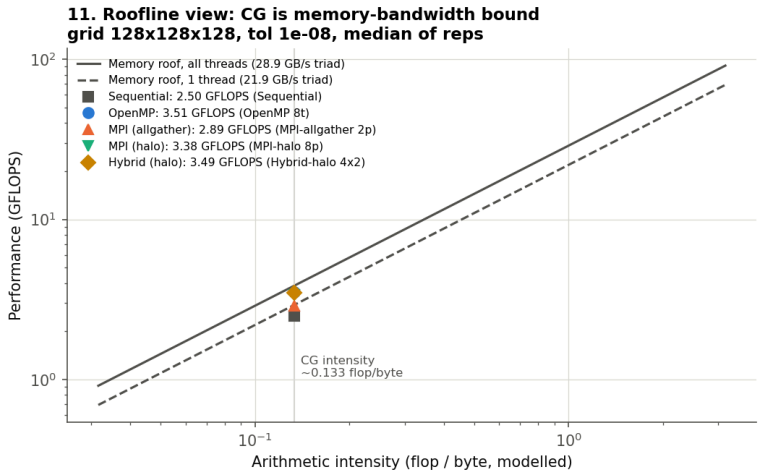


Figure 3. Roofline: performance vs the memory-bandwidth limit.

> Measured memory bandwidth grows only 1.32× from 1 to 8 cores (21.9 → 28.9 GB/s). CG reads a lot of data for very little arithmetic, so it runs at about **90% of the memory limit**. The bottleneck is the memory, not the code.

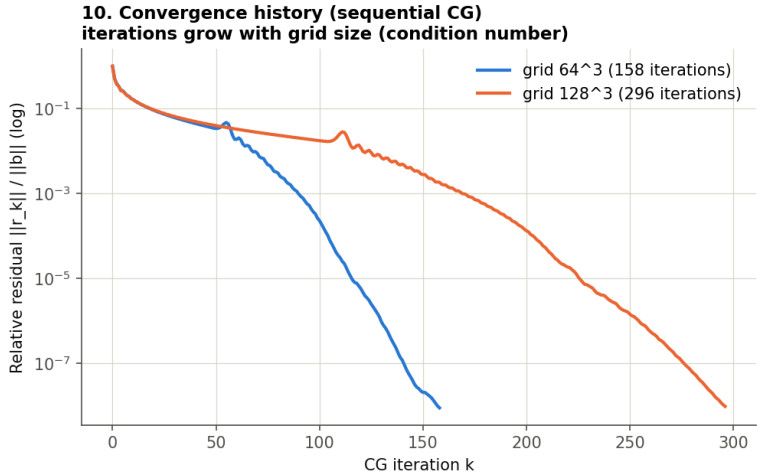


Figure 4. Error reduction per iteration for two grid sizes.

> The solver converges reliably; larger grids need more iterations (158 for 64^3, 296 for 128^3), as expected from theory.

6 Conclusion

- We implemented and verified four parallel versions of the Conjugate Gradient solver; all produce identical results.
- Speedup is limited to about 1.4× on 8 cores because memory bandwidth, not processing power, is the bottleneck.
- Sending only the needed data between MPI processes (halo exchange) is 1.69× faster than sending everything.
- This confirms the idea behind HPCG: for sparse problems, memory speed matters more than arithmetic speed.

Note: this is an educational, HPCG-inspired implementation, not the official HPCG benchmark. Code and raw data: github.com/Anuttaman216/HPC_FInal_Lab_Project

References

1. J. Dongarra, M. A. Heroux, P. Luszczek, "High-performance conjugate-gradient benchmark: A new metric for ranking high-performance computing systems," *Int. J. High Performance Computing Applications*, 30(1), 2016.
2. M. R. Hestenes, E. Stiefel, "Methods of conjugate gradients for solving linear systems," *J. Res. Nat. Bur. Standards*, 49(6), 1952.
3. S. Williams, A. Waterman, D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Commun. ACM*, 52(4), 2009.
4. Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed., SIAM, 2003.